IT1244 Project Report: Machine Learning Modeling of Transformer Oil Temperature

Chin Yan Zu, Eugene Tan Jun Yuan, Emmanuel Ng Teng Yang, Jeremy Lee Zhe

Introduction

Transformers are critical components in our electrical systems, helping grids and appliances to control the power distributed throughout the grid. Oil in transformers act as a coolant, regulating temperature to prevent overheating. Hence, the temperature of the oil coolant represents a way to track transformer performance and detect issues as they arise. Since oil temperature is a good proxy for transformer conditions, we use machine learning models to forecast future oil temperatures, giving us a good gauge for the condition of the transformer. In this report, we outline 4 models and evaulate their relative performances in predicting oil temperature..

Current Literature

An approach outlined by Huang et al. (2025) is to use a hybrid ARIMA-LSTM-XGBoost Model. This is effective as ARIMA can capture the linear trends while Long Short-Term Memory network (LSTM) and XGBoost can capture non-linear trends. However, the hybrid model is computationally intensive, making it unfeasible for this project, though we can consider the models that it is built with.

Bai et al. (2018) outlines the efficacy of time series models, LSTM, XGBoost and ARIMA are also evaluated as well as Temporal Convolution Networks (TCN), showing that TCN models consistently outperform canonical recurrent architectures like LSTM and GRU across a variety of sequence modelling tasks.

Dataset Analysis

For this problem, we are given time series datasets of 2 transformers from 1st July 2018 to 26th June 2020. Each time point includes the target variable oil temperature, as well as 6 other features: HUFL, High Voltage Useful Load (active power on high voltage side); HULL, High Voltage Useless Load (reactive power on high voltage side); MUFL, Medium Voltage Useful Load (active power on medium voltage side); MULL, Medium Voltage Useless Load (reactive power on medium voltage side); LUFL, Low Voltage Useful Load (active power on low voltage side); LULL, Low Voltage Useless Load (reactive power on low voltage side).

Doing some cursory data exploration, the dataset does not have any missing values or missing time points. However, the timeframe of the data is only 2 years, which might be too little for our models to learn more complex patterns about the data. Additionally, the problem also restricts us from using oil temperature as an input feature, preventing the use of autoregressive models like ARIMA.

Feature Visualization and Selection

Firstly, we created a time series visualization of oil temperatures of both transformers.

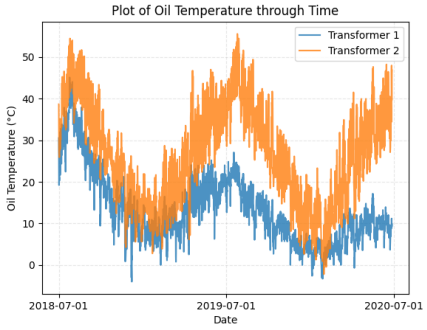

Figure 1: Time Series Plot of Oil Temperature in both transformers.

The 2 transformers generally have the same trend, but transformer 2 reaches higher temperatures. However, due to the short timeframe, it is hard to definitively draw conclusions about the seasonality or patterns of the data. We then plot the other features with respect to time.

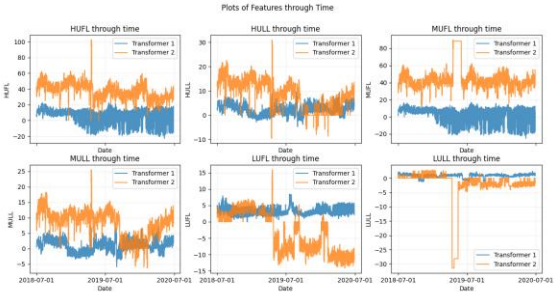

Figure 2: Time Series Plots of HUFL, HULL, HUFL, MULL, LUFL, and LULL in both transformers.

Unlike oil temperature, none of the other features show a clear trend through time. Lastly, to investigate the relationship between each feature, we created a correlation heatmap.

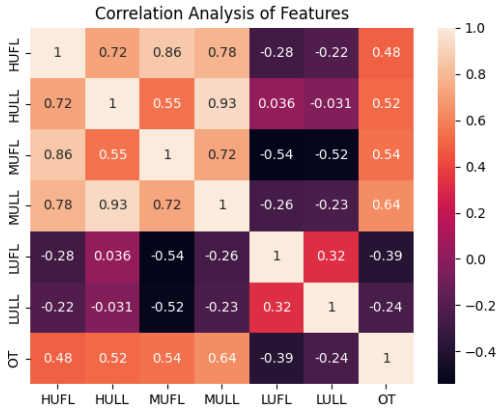

Figure 3: Correlation Heatmap of Features

While there are some features with low correlations to oil temperature, our deep learning model can learn complex patterns that might not be captured by correlation coefficients. Hence, we focus on pruning features with high correlation to other features. HUFL has a high correlation with HULL and MUFL at 0.72 and 0.86 respectively, while MULL has a high correlation with HULL and MUFL at 0.93 and 0.72. Hence, we prune HUFL and MULL and select HULL, MUFL, LULL and LUFL as our input features for the rest of the project.

Data Preparation

We merged the data of the 2 transformers and shifted columns of oil temperature back to create oil temperatures 1 hour, 1 day, and 1 week for every time point. We then split the data into time point groups of 1 week and randomly chose 20% of the time point groups to be our test set while the rest would be our training set. This was done to preserve the time series nature of our data while still having the training and testing set cover a wider portion of our data. After, we performed mean normalization on the columns of both the training and testing set with respect to the training set (using the mean, maximum and minimum of the training set to do calculation on both sets). We also created a function to help automatically create time windows that takes into account the missing time point groups after our train-test split.

Methodology

We are asked to consider 3 scenarios, predicting oil temperatures 1 hour (scenario 1), 1 day (scenario 2), and 1 week (scenario 3) in advance. Additionally, we are not allowed to use oil temperature at any time point. We will be approaching this using 4 models, Linear Regression, Extreme Gradient Boosting (XGBoost), Long Short-Term Memory (LSTM), and Temporal Convolution Network (TCN). Our method pipeline is illustrated below.

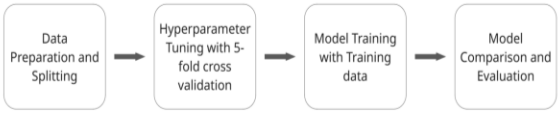

Figure 4: Method Pipeline for Model Creation and Comparison

We start by tuning each model with hyperparameters, using grid search for models with less parameters and Bayesian search for models with more hyperparameters to reduce runtime. Afterwards, models will be trained and tested on unseen data to evaluate their relative performance via their Root Mean Square Error (RMSE).

Linear Regression

Our baseline model for this project is linear regression. Linear regression is a traditional machine learning method that finds the linear relationship between dependent and independent variables. Since time series data breaks the assumption that errors are independently and identically distributed (IID), it is generally not considered suitable for time series forecasting and is not expected to perform well. However, Toner and Darlow (2024) found that it performed well at time series forecasting despite its simplicity, making it worth experimenting with.

Since linear regression itself has no hyperparameters to tune, we primarily focused on tuning the lookback window. We implemented the lookback window by adding the lagged features as input, and we tuned it by performing 5-fold cross validation on our training data. We then chose lookback windows of 1 for scenario 1, 4 for scenario 2 and 240 for scenario 3 as they achieved the best RMSE in testing. We then fitted the linear regression models on our training set with the specified lookback windows to create our linear regression models.

Extreme Gradient Boosting

Gradient boosting is an ensemble method that builds a strong model by combining many weak decision trees (Chen and Guestrin 2016). Each decision tree is built sequentially to predict the residuals of the previous decision tree. This is a form of gradient descent algorithm, resulting in the name

gradient boosting. Extreme Gradient Boosting (XGBoost) is a form of gradient boosting built for scalability and speed through parallel computation, resulting in better performance than regular Gradient Boosting Machines. Additionally, it utilizes regularization, shrinkage, and column subsampling to help prevent overfitting (Chen and Guestrin 2016). Hence, XGBoost can capture complex non-linear relationships, making it suitable for time series forecasting.

As XGBoost has many hyperparameters to better tune the different steps of the model training, it would be too time-consuming to achieve the best possible set of hyperparameters. Hence, we opted for RandomizedSearchCV on a regular range on each hyperparameter as well as tested a range for the lookback window. This helps to test randomly selected sets of hyperparameters for each lookback window and return the best set from the random selection. After tuning, the lookback window that achieved the best RMSE was 2 for scenario 1, 40 for scenario 2, and 168 for scenario 3.

Long Short-Term Memory

A LSTM model is a type of Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data, making it suitable for use in time series forecasting. Unlike traditional RNNs, LSTM models contain memory cells with 3 gates: Input Gates, which controls the addition of new information, Forget Gates, which controls how much of the information is removed, and Output Gates, which control the output of the cell (Huang et al., 2025). This allows the model to retain information from further back while discarding unnecessary information, preventing the vanishing and exploding gradient problems faced by traditional RNNs. Therefore, LSTM models can learn complex long-term patterns, allowing them to forecast time series data such as oil temperature.

Due to the long runtime for LSTM models, we decided to primarily focus on tuning the lookback window for each scenario, increasing the range of values for each scenario to consider the longer-term dependencies required for predictions further in the future. We tuned the lookback window by performing 5-fold cross validation on our training data, choosing which lookback window to use by comparing their average RMSE across the folds. After tuning, the lookback windows that achieved the best RMSE were 2 for scenario 1, 4 for scenario 2 and 240 for scenario 3.

We then trained our LSTM model using the selected lookback windows. For the model, we used a one-layer LSTM with 64 cells, a learning rate of 0.0001 and a batch size of 32 based on convention and concerns with our runtime. We also added early stopping with a patience of 20 to prevent over-fitting. We then fitted the model to our training set with 300 epochs.

Temporal Convolution Network

Temporal Convolution Networks (TCN) are a type of Convolutional Neural Networks (CNN) adapted for use on sequential data. This makes TCNs suitable for time series forecasting tasks. A TCN model takes a series of input data and passes them through a set of convolution layers to capture temporal correlations. The main difference between a TCN model and a traditional CNN model is in the adaptation of causal convolutions, where each output depends only on data from past and present time points, ensuring that the model does not train on future data. We also use dilated convolutions which skip over certain points in the input data to allow our model to capture longer temporal dependencies.

For the TCN model, it is critical that it maintains a receptive field large enough to capture the entirety of the input data supplied. Hence, we tune hyperparameters filters, kernel size (k), dilation base (b) and input data length (l), of which the latter three primarily affect this receptive field (Lässig 2021). Input data length is directly related to our lookback window. The number of convolutional layers (n) necessary can then be found from the following formula (Lässig 2021) to ensure that our TCN model achieves full history coverage.

$$n = \left\lceil log_b \left(\frac{(l-1) \cdot (b-1)}{(k-1)} + 1 \right) \right\rceil$$

Figure 5: Formula for Computing the Number of Convolutional Layers Needed

We opted for a Bayesian search method on a selected range of values for each hyperparameter while adhering to the formula. After tuning, the best set of hyperparameters (lookback window, filters, kernel size, dilation base, number of layers) that achieved the best RMSE was (6, 64, 2, 3, 3) for scenario 1, (14, 64, 2, 2, 4) for scenario 2, and (240, 64, 7, 3, 4) for scenario 3 where we also specifically set batch size to 128 to speed up computation.

We then found that lowering our learning rate from 0.001 to 0.000001 and increasing batch size from 32 to 64 greatly improved convergence times and reduced model overfitting. Lowering learning rates while automatically halving learning rate during training when model performance stagnated dramatically improved model performance.

Results and Discussion

After training the various models, we evaluated them on the testing set using root mean square error (RMSE). We then compiled the results into tables for each scenario for comparison. As stated, we expected linear regression to perform

the worst, but surprisingly across the 3 scenarios it had the best general performance, with other models performing at similar accuracy.

1-hour forecasting

Model	RMSE	Lookback N
Linear Regression	9.7699	1
XGBoost	10.3329	2
LSTM	10.0067	2
TCN	10.4587	6

Table 1: Result for Scenario 1

1-day forecasting

Model	RMSE	Lookback N
Linear Regression	9.7610	4
XGBoost	8.3254	40
LSTM	9.9455	4
TCN	9.9007	14

Table 2: Result for Scenario 2

1-week forecasting

Model	RMSE	Lookback N
Linear Regression	9.8688	240
XGBoost	11.2424	168
LSTM	10.1524	240
TCN	10.0292	240

Table 3: Result for Scenario 3

We suspect that due to the slicing of data into time point groups and selecting random ones for the testing set, the temporal structure of our dataset was disrupted. This resulted in data far closer to being identically and independently distributed which negatively affected XGboost, TCN and LSTM whose features which account for longer term temporal trends and seasonality were penalized while simpler models like linear regression gain a significant boost.

The highly linearly correlated nature of the dataset also benefited models like linear regression. While the worst offenders were removed, there was still a remarkably high correlation between both the non-UU features on top of a fairly high correlation with the output feature. Together, these features likely resulted in the strong performance displayed by linear regression.

The only outlier to this performance is XGBoost in 1 day forecasting with a staggering 40 for lookback windows. In 1 day and 1 week forecasting higher lookback windows generally showed better performances than models which used smaller lookback windows, underlying a possible correlation between a larger lookback window and better model performance for longer lag time predictions.

Conclusion

Our results show the importance of data preparation and how given different data preparation conditions and methodologies different models thrive. It also underlines the importance of picking the correct model for the task to best exploit its strengths and weaknesses. While the initial hypothesis was that linear regression would perform poorly due to general time series conditions, the formulation of the experimental conditions created an ideal environment for it to perform.

There is also further room for evaluating combination models such as the hybrid ARIMA-LSTM-XGBoost (Huang et al., 2025), which significantly outperformed our models with an RMSE of 0.9954 with a similar dataset. This shows the hybrid models' ability to capture linear and nonlinear relationships, which greatly boosts their ability to forecast complex time series data.

Oil temperature forecasting affects workers managing electrical grids and society at large. It is a task uniquely suited to machine learning and the most successful models at predicting oil temperature are complex hybrid or deep learning models, which are both difficult to interpret and computationally intensive. If computational cheaper algorithms can perform comparably to more sophisticated models while producing predictions that are more interpretable, they would allow for better and safer judgement to be made by workers, while being more environmentally friendly, hence allowing us to keep the utility of predicting transformer conditions without the social costs.

References

Bai, Shaojie; Kolter, J. Zico; and Koltun, Vladlen. 2018. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv preprint arXiv:1803.01271.

Chen, T. and Guestrin, C. 2016. XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 785–794. doi.org/10.1145/2939672.2939785.

Huang, X.; Zhuang, X.; Tian, F.; Niu, Z.; Chen, Y.; Zhou, Q.; and Chao, Y. (2025). A Hybrid ARIMA-LSTM-XGBoost Model with Linear Regression Stacking for Transformer Oil Temperature Prediction. Energies 18(6), 1432. doi.org/10.3390/en18061432.

Lässig, F. 2021. Temporal Convolutional Networks and Forecasting. Unit8. Available at: https://unit8.com/resources/temporal-convolutional-networks-and-forecasting/.

Toner, W., Darlow, L. 2024. An Analysis of Linear Time Series Forecasting Models. arXiv:2403.14587. doi.org/10.48550/arxiv.2403.14587.

Yadav, A. 2024. Temporal Convolutional Network — An Overview. Medium (Oct. 9). Available at: https://medium.com/@amit25173/temporal-convolutional-network-an-overview-4d2b6f03d6f8.